Agentic AI: Towards Robust GUI Agents for Real-World Applications

GUI Agent Tuning for Interface Prediction

Jimin Go
Computer Engineering
Arizona State University
Tempe, AZ, United States
jimingo1@asu.edu

Michael Glass
Computer Science
Arizona State University
Tempe, AZ, United States
mjglass2@asu.edu

Neil Shenoy
Computer Science
Arizona State University
Tempe, AZ, United States
nsshenoy@asu.edu

Mohana Samara Simha Reddy Attunuru
Computer Science
Arizona State University
Tempe, AZ, United States
mattunu1@asu.edu

ABSTRACT

Graphical User Interfaces (GUI) are a new focus of machine learning, and next action prediction is necessary for enhancing user throughput for better GUIs. By creating a model that is able to accurately predict a user's next action on a GUI, we expect that we can improve the user experience and reduce the time required to find interface components on a generalized set of applications. In this project, we design and assess various machine learning models, specifically Convolution Neural Networks (CNN), Gate Recurrent Units (GRU), Transformers, and Long Short-Term Memory Networks (LSTM). Through experimentation, we investigate each model to discover optimal parameters that yield the highest success rate. To do this, we converted a dataset of various elements, actions, and screen transitions into token-based sequences to allow a model to train and predict the average user workflow.

Success in our project was defined by the metrics of validation accuracy, mean reciprocal rank (MRR), and hit rate at K (H@K). Through extensive experimentation, models like a one dimensional CNN proved to surpass the other complex models, and current parameter and hyperparameter tuning have resulted in the highest accuracy so far at 0.5598 and highest MRR value of 0.7173. Therefore, we have chosen to continue research utilizing the CNN model, and as we push forward with data cleaning and parameter tuning, our model is improving towards the goalpost of a 0.70 success rate for predictions. Our research shows that GUI agents succeed at the highest rate utilizing lightweight, simple models that follow sequence based data to best emulate human-computer interaction on a variety of different GUIs.

CCS Concepts

• Computing methodologies → Machine learning → Learning paradigms → Reinforcement learning → Sequential decision making; • Human-centered computing → Human computer interaction (HCI) → HCI design and evaluation methods

KEYWORDS

Graphical User Interfaces, Sequence Models, Transformer Language Models, Convolution Neural Network, GUI Prediction Optimization

ACM Reference format:

Jimin Go, Michael Glass, Mohana Samara Simha Reddy Attunuru, and Neil Shenoy. 2025. Agentic AI: Towards Robust GUI Agents for Real World Applications: GUI Agent Tuning for Interface Prediction. *CSE 475 Final Project*. ACM, New York, NY, USA, 8 pages.

1. Introduction

Graphical User Interfaces are an incredibly important part of human-computer interactions, and during this period of increased machine learning development, boundaries are being pushed in regards to training done on GUIs. As more models become available, the importance of heterogeneous models being correctly trained and generalized across a variety of different applications is becoming quickly apparent. Models must be able to generalize across unique applications, or even screens in the same applications, to allow a model to correctly identify user workflows and how to optimize them. Varying layouts can oftentimes lead to

issues for models, which is problematic for launching applications that utilize separate formatting across the entire system. Current models face heavy challenges due to this, and applications utilizing different elements, actions, and screen types cause significant noise in training. This makes it difficult to create predictive power. Creating a model that is able to generalize across applications would be a significant breakthrough that allows for GUIs to improve at the individual level, allowing for changes based on a user's habits during their time spent on an application.

Our research to solve this specific problem explores optimization of different model types, parameters, and hyperparameters in order to find the best strategies to generalize GUIs and predict a user's path through a variety of applications. Specifically, we examined the mean reciprocal rank and validation accuracy of transformer, CNN, LSTM, and GRU models with various different parameters and hyperparameters.

2. Related Work

Graphical User Interface automation has evolved significantly over the past decade, progressing from rule-based testing frameworks to vision-based element detection systems, and more recently to foundation model-driven intelligent agents. A central challenge throughout this evolution has been enabling systems to understand and predict user interactions across diverse applications with varying layouts, elements, and workflows. Early work focused primarily on GUI testing and element detection, establishing the perception capabilities necessary for automated GUI understanding. As the field matured, researchers began exploring predictive modeling of user behavior and developing large-scale agent systems capable of reasoning about multi-step tasks. More recently, questions have emerged about the optimal model architectures for specific GUI prediction tasks, particularly regarding the trade-offs between model complexity and efficiency. In this section, we review relevant work across five key areas: early GUI testing frameworks and element detection (Section 2.1), modern GUI agents and foundation models (Section 2.2), user behavior prediction in GUI contexts (Section 2.3), sequence modeling architectures for structured prediction (Section 2.4), and approaches to generalization and context-aware learning (Section 2.5).

2.1 Data Preprocessing & Representation

The foundations of automated GUI understanding were established through early testing frameworks and element detection systems that sought to reduce manual effort in software validation. Nguyen et al. [19] introduced GUITAR, an innovative tool for automated testing of GUI-driven software that employed a hybrid capture-and-replay approach. GUITAR combined scripted testing with exploratory random-walk traversal to achieve broader coverage of GUI states and transitions. This work demonstrated how automation could significantly reduce repetitive effort in GUI testing and provided an early example of bridging manual and automated methods. However, GUITAR was fundamentally designed for test automation rather than predictive modeling of user behavior. The system focused on replaying recorded interactions and exploring application states systematically, but lacked structural or vision-based understanding of GUI elements that would enable reasoning about future user actions. This limitation meant that while GUITAR could verify whether specific interaction sequences functioned correctly, it could not predict what a user might do next based on their current context.

Building on the need for better GUI understanding, Chen et al. [20] conducted a large-scale empirical study comparing traditional image-processing techniques with modern deep learning methods for GUI element detection. Their study analyzed approximately 50,000 GUI images to systematically evaluate different approaches to identifying and localizing interface components. They found that deep learning methods significantly outperformed traditional computer vision techniques, particularly when handling GUI-specific perception challenges such as dense layouts, overlapping elements, and visual similarity between different component types. To address these challenges, Chen et al. proposed a coarse-to-fine detection method for non-text elements combined with a CNN-based classifier specifically designed for text components. Their work provided strong empirical evidence that neural networks could learn robust visual representations of GUI elements across diverse applications. Despite these advances in perception, their approach remained focused solely on detecting what elements exist on a screen rather than reasoning about how users might interact with them. Furthermore, their dataset lacked the contextual information necessary for action prediction, such as DOM tree structures or sequences of user interactions, limiting its applicability to predictive modeling tasks.

While these foundational works successfully established core perception capabilities for GUI understanding—enabling systems to identify and localize interface elements reliably—they left a critical gap in the automation pipeline. Perception alone is insufficient for creating intelligent GUI agents; systems must also reason about user intentions and predict likely interaction sequences. This gap motivated subsequent research

toward end-to-end GUI agents capable of both perceiving interface elements and reasoning about actions, as we discuss in the following section.

2.2 Modern GUI Agents and Foundation Models

As the field progressed beyond basic perception and testing, researchers began developing comprehensive agent systems capable of end-to-end GUI automation. Nguyen et al. [3] provided a comprehensive survey tracking the evolution of GUI automation from rule-based methods through vision-based approaches to modern foundation-model-driven agents. Their survey established a four-component framework for GUI agents: (1) Perception for visual understanding of GUI elements; (2) Reasoning for interpreting task requirements and context; (3) Planning for generating multi-step action sequences; and (4) Acting for executing interface actions. This taxonomy connects historical methods to contemporary architectures and identifies generalization across diverse applications as the field's central open problem. However, the survey lacks empirical analysis of efficiency considerations, latency constraints, or lightweight approaches to specific prediction subtasks. Our work focuses specifically on the reasoning component—predicting next elements in GUI interactions—using efficient architectures suitable for resource-constrained environments.

The most ambitious recent effort in GUI automation is OS-Atlas, developed by Wu et al. [26] as a foundation action model for generalist GUI agents. OS-Atlas introduces the largest cross-platform GUI grounding corpus to date with over 13 million GUI elements across 2.3 million screenshots spanning Windows, Linux, MacOS, Android, and web platforms. The model employs two-phase training: GUI grounding pre-training for element localization, followed by action fine-tuning for learning appropriate interactions. A key innovation is its unified action space, which resolves cross-platform naming conflicts (e.g., standardizing "tap" and "click"), reducing action types from 17 to 10 and enabling better knowledge transfer. Wu et al. also developed the first comprehensive multi-platform data synthesis toolkit for automated training data collection. OS-Atlas demonstrates significant improvements over previous state-of-the-art across six benchmarks, establishing itself as a viable open-source alternative to commercial models like GPT-4o.

Despite these results, OS-Atlas faces limitations that motivate our work. First, it relies on proprietary GPT-4o for instruction grounding data annotation, creating cost barriers and limiting reproducibility. Second, it still lags human performance significantly (25% vs. 75% success rate on OSWorld). Third, its scale makes it unsuitable for on-device deployment—the large vision-language architecture and extensive training data (though desktop data remained limited at only 54,000 screenshots versus 1.9 million for web) necessitate substantial computational resources. Our work explores a fundamentally different approach: rather than building a comprehensive foundation model, we focus on next-element prediction using lightweight architectures with token-based representation instead of vision-language processing, aiming for competitive performance with significantly lower computational requirements suitable for efficient, on-device deployment.

2.3 User Behavior Prediction in GUI Contexts

While foundation models pursue comprehensive GUI understanding, parallel research has explored focused approaches to predicting user behavior. Zhou and Li [21] analyzed over 20 million mobile user clicks, achieving 48% top-1 accuracy (exact next click prediction) and 71% top-3 accuracy (correct element in top three predictions). They compared against nine baselines spanning heuristics-based, classic machine learning, and deep learning methods, outperforming all approaches. Critically, Zhou and Li identified that model performance degrades significantly as screen complexity increases—more interactive elements make prediction substantially more difficult. This finding directly impacts our work, as we must also contend with varying element density across applications and screen types. While their work demonstrates prediction viability, it remained mobile-focused and did not systematically explore how different sequence architectures affect performance, motivating our comparative study.

Our primary baseline is AutoDroid-V2 [1], which reports 67.1% success rate on GUI task completion. AutoDroid-V2 represents the current state-of-the-art for our problem domain, and our dataset derives directly from its publicly available interaction logs. The system employs small language model (SLM)-based agents using code generation for GUI automation. This success rate serves as our target performance threshold—matching or exceeding this level would demonstrate competitive results despite using simpler, more efficient architectures. While AutoDroid-V2 uses code generation, our token-based sequence modeling offers potential advantages in inference

speed and computational efficiency, particularly for resource-constrained environments where code generation and execution may introduce unacceptable latency.

2.4 Sequence Modeling Architectures

Transformer architectures have achieved remarkable success across sequence modeling domains, as documented by Lin et al. [4]. Their self-attention mechanism enables long-range dependency modeling, driving state-of-the-art results in NLP, vision, and multimodal learning. Popel and Bojar [22] provided Transformer training guidance through Tensor2Tensor, offering hyperparameter tuning insights, though with caveats about hardware requirements and domain-specific applicability. Amatriain et al. [23] cataloged Transformer evolution, documenting variants and pretraining strategies but offering limited guidance on architecture selection for specific tasks.

Despite Transformer dominance, evidence suggests architectural complexity should match data characteristics. For short sequences with local dependencies, simpler models may prove more effective. Kim [8] demonstrated CNNs achieve strong sentence classification performance using 1D convolutional filters with kernel sizes {2, 3, 4} to capture local n-gram patterns—our 1D-CNN directly follows this design. Johnson and Zhang [5] established that CNNs effectively leverage word order for text categorization and often outperform complex models on short-text tasks with limited context. This directly relates to GUI prediction: if interaction patterns depend primarily on local structure (e.g., "element → action → element") rather than long-range dependencies, convolutional architectures may better suit the task.

Recurrent architectures offer another alternative. Tang et al. [6] demonstrated GRUs effectively capture sequential dependencies for sentiment classification, providing a lighter alternative to LSTMs through simplified gating with fewer parameters and faster convergence. Gupta et al. [24] systematically compared LSTM variants (vanilla, stacked, bidirectional) for time series prediction, finding stacked LSTM outperformed others. While focused on time series rather than discrete GUI events, their work reinforces the importance of systematic architectural comparison. We implement both standard LSTM and GRU to evaluate recurrent processing against convolutional and attention-based approaches.

These findings inform our central hypothesis: GUI interactions exhibit primarily local, short-range dependencies rather than long-range relationships that motivate Transformers. Users typically interact with nearby or functionally related elements, with context naturally bounded by visible screen contents. Unlike natural language requiring integration across many sentences, GUI interactions involve more constrained decision spaces and shorter context windows. We predict simpler models—particularly CNNs with strong local pattern recognition bias—may match or exceed Transformer performance while offering significant computational efficiency. This motivates our systematic comparative study across CNN, LSTM, GRU, and Transformer architectures under identical conditions.

2.5 Generalization and Context-Aware Learning

Regardless of architecture choice, achieving robust generalization to unseen applications remains fundamental. Tutum et al. [25] addressed this through their Context+Skill architecture, explicitly separating contextual understanding from reactive skill execution. Their approach uses three components: a Context LSTM module integrating temporal and environmental information, a Skill feedforward module learning reactive patterns, and a Controller network combining both for final actions. The key innovation is that both modules learn to automatically standardize outputs across contexts—this standardization emerges from training rather than explicit design, simplifying the Controller's task in novel situations. They demonstrated this across Flappy Bird, Lunar Lander, and CARLA, testing generalization by extending parameter ranges 35-75% beyond training distribution with multi-parameter variations. Context+Skill consistently outperformed context-only or skill-only baselines, validating explicit architectural separation for generalization.

However, their approach has limitations informing alternative designs. They require fixed topologies with hand-crafted features rather than learning from raw input, limiting flexibility. More significantly, neuroevolution with NSGA-II multi-objective optimization creates substantial computational costs—only single CARLA runs were feasible. While demonstrating generalization in game environments, transfer to complex real-world tasks remains unclear.

Our work shares their generalization focus but pursues a different approach. Rather than explicit Context and Skill modules, we rely on architectural inductive biases—particularly CNN local pattern recognition—combined with token-based representation. Our preprocessing transforms interactions into abstract element-action-element sequences, removing application-specific visuals while preserving structural patterns. This abstraction serves similar purposes to Tutum et al.'s learned standardization but achieves it through data

representation rather than architectural separation. We rigorously test cross-application generalization using disjoint application splits: train, validation, and test sets contain completely different applications, ensuring models must generalize rather than memorize application-specific patterns. This parallels Tutum et al.'s out-of-distribution testing but for GUI interactions.

The related work reveals several critical gaps motivating our research. Modern GUI agents like OS-Atlas [26] achieve impressive results but require massive computational resources unsuitable for on-device deployment. No systematic comparison exists specifically for GUI next-element prediction, leaving uncertainty about optimal architectural properties. Prior work primarily relies on image-based representations [20] or large vision-language models [26], leaving token-based sequence modeling underexplored. While generalization has been identified as central [3], few studies employ rigorous evaluation with strictly disjoint application splits.

3 Data & Methods

In this section, we outline the experimental design used to evaluate the proposed GUI interaction modeling approach. We describe the data preprocessing pipeline and representation scheme, explain the rationale behind our choice of model architecture, and detail the training setup, evaluation metrics, and comparative results across baseline models.

3.1 Data Preprocessing & Representation

A key component of this work is the data preprocessing pipeline, which reformulates raw GUI interaction logs into a structured, token-based representation suitable for sequence modeling. The raw dataset was derived from the publicly-available GUI interaction logs collected by the AutoDroid-V2 project [1]. Each JSON file contains hierarchical information about screens, interface elements, and recorded user actions such as taps or navigations.

Unlike prior works that directly use image-based or DOM-tree representations of GUI states [2, 3], our preprocessing method treats each GUI transition as a semantic sequence of discrete symbols capturing the underlying action logic. Each GUI path was decomposed into (element–action–element) triplets, where each sample takes the form (element_src, action) → element_dst. This formulation allows the model to learn relational reasoning between interface components, essentially predicting what element will be activated next given the current element and user action.

To ensure high-quality supervision, unresolved transitions (e.g., cases labeled as dst_not_found or no_widget) were filtered out. We also constructed autoregressive sequences like [ELEMENT_src, ACTION_1, ELEMENT_1, ACTION_2, ELEMENT_2, …], enabling optional language-model–style auxiliary training.

All tokens (elements, actions, and screens) were assigned unique identifiers, and a unified vocabulary was created with special tokens [PAD], [UNK], [CLS], and [SEP]. This representation transforms complex GUI behaviors into structured token sequences, allowing direct application of text-style sequence models such as Transformers or CNNs.
In this sense, the preprocessing pipeline itself provides a novel intermediate representation of GUI interactions, bridging symbolic GUI reasoning and sequence-based neural modeling.

3.2 Model Selection

Although Transformer-based architectures have recently dominated many sequence modeling tasks [4, 5], we questioned whether such complex models are always necessary for short and structurally repetitive sequences such as GUI interaction paths.
Specifically, we hypothesized that simpler models, such as Convolutional Neural Networks (CNNs) or recurrent networks, might actually perform better, since GUI transitions often depend on local patterns (e.g., “element → action → element”) rather than long-range dependencies. This observation aligns with findings from natural language processing, where CNNs and RNNs frequently outperform Transformers in short-text or limited-context scenarios [5,6,7].

Based on this reasoning, we decided to conduct comparative experiments using multiple architectures (e.g., CNN, LSTM, GRU, and Transformer) to empirically verify whether a simpler model could more effectively capture GUI interaction logic.
Among these, the 1D-CNN [8] demonstrated the strongest performance. CNNs can efficiently capture local sequential dependencies through convolutional filters, which naturally align with the short-range relational structures of GUI element–action transitions. In contrast, the Transformer’s global self-attention introduces unnecessary complexity and may lead to overfitting when the data lacks long contextual relationships.

These results suggest that, for structured GUI reasoning tasks, simplicity and locality-awareness can sometimes be more beneficial than global attention mechanisms,

supporting the idea that architectural complexity should be matched to the structural characteristics of the data.

3.3 Experiment Settings

3.3.1 Model Implementation

All experiments were implemented in PyTorch using GPU acceleration. The dataset was split into train, validation, and test sets using a disjoint application rule to evaluate cross-app generalization, consistent with the AutoDroid-V2 setup [1]. Each input sequence was tokenized as [CLS] + tokens + [SEP], padded or truncated to a maximum length of 32, and accompanied by token-type embeddings distinguishing between elements, actions, and screens. The embedding dimension was set to 256, which balances representational capacity and computational efficiency, as suggested by prior GUI sequence models [9].

3.3.2 Training Configuration

All models shared a unified optimization setting for fairness of comparison. We used the AdamW optimizer [11] with an initial learning rate of 1×10^{-4}, weight decay = 3×10^{-4}, and a cosine learning-rate schedule with linear warm-up [12] to ensure stable early-phase convergence. Label smoothing ($\varepsilon = 0.05$) was applied to the classification loss to mitigate over-confidence [13]. Each model was trained for up to 500 epochs with a batch size of 64, and early stopping was triggered if the validation MRR did not improve for 30 epochs. The hidden dimension (d_model) was set to 512, the feed-forward network (FFN) dimension to 1024, and dropout to 0.3. The Transformer model consisted of 4 encoder layers with 4 attention heads, while all other models were configured to have comparable capacity. Gradients were clipped to a maximum L_2 norm of 1.0 to prevent exploding gradients during training. All experiments were conducted with a fixed random seed for reproducibility and performed on a single GPU.

3.3.3 Model Variants

To test our hypothesis regarding local versus global modeling capability, we implemented and compared the following architectures:

1D-CNN [8]: Applies convolutional filters with kernel sizes {2, 3, 4} across token embeddings to capture local n-gram-like dependencies. Each filter output is aggregated via global max-pooling, followed by a two-layer feed-forward classifier.

LSTM [14]: Processes the sequence recurrently to capture order-sensitive dependencies and long-term context through gated cell memory.

GRU [15]: A simplified gated variant of LSTM with fewer parameters and faster convergence.

Transformer Encoder [16]: Consists of three encoder layers, each with four self-attention heads and a feed-forward dimension of 512, designed to model global contextual dependencies.

Bag-of-Embeddings (BoE) [17]: Serves as a non-sequential baseline that averages token embeddings and classifies the pooled representation.

3.3.4 Evaluation Metrics

Model performance was evaluated using multiple complementary metrics to capture both accuracy and ranking quality:

Accuracy: Measure the proportion of cases where the model's top 1 prediction matches the ground-truth destination element.

Mean Reciprocal Rank (MRR): Evaluates how highly the correct label is ranked among all possible candidates, placing emphasis on early-rank correctness [18].

Hits@k (k=3,5,10): Denotes the fraction of test samples where the correct element appears within the model's top-k predictions. This metric is widely used in link prediction and retrieval tasks [19] and offers a more lenient measure of ranking accuracy.

3.3.5 Initial findings

Our early experiments show that simple sequence models such as 1D-CNN, LSTM, and GRU outperform Transformers on our GUI transition prediction task. This suggests that GUI interaction paths primarily rely on local, short-range dependencies (e.g., "element → action → element"), making complex self-attention architectures less advantageous. We also found that our token-based preprocessing pipeline (element/action abstraction + removal of unresolved transitions) noticeably stabilized training and improved label consistency across models. Overall, these findings indicate that architectural simplicity with strong local inductive bias is more suitable for GUI next-element prediction than large Transformers, guiding our choice to adopt CNN as the primary model for further experiments.

4. Results

Current exploration on models is primarily working through an assortment of models that we believe will retrieve the best results through parameter optimization and improvements in data cleaning. Results were collected on testing across 500 Epochs with a batch size of 256, dropout

of 0.1, d_model of 256, 10 layers, and a n-head of 4 across all models being tested.

4.1 Baseline

The baseline metric we are utilizing for our data is the AutoDroid-V2 model, which reports a success rate of 0.671 [1]. Our model is working on a similar problem to the AutoDroid-V2 model, and our dataset is drawn from the data they collected, so achieving a similar or higher success rate for our model defines success for our research. Current solutions have slowly been approaching this success rate, and metrics we track help us understand how to increase our success rate towards the AutoDroid-V2 model's success rate.

4.2 Tracked Metrics

The metrics we tracked through our research on various models were validation accuracy, mean reciprocal rank, and hit rate at K. Validation accuracy is essential for this model, as generalization is the primary metric of success for our problem. Mean reciprocal rank (MRR) is important to our model because of how many options the model has to choose from in a variety of different contexts, so the speed in which it is able to select the primary next element is vital. Additionally, hit rate at K (H@K) also applies in a similar manner to MRR, as it tracks how many users the model successfully ranks at least one relevant element in the top K recommendations.

4.3 Quantitative Results

The following tables explains the results of testing across the 5 models explored during our experimentation. Initial research focused on finding the greatest values across the metrics in order to further pursue research on a specific model with the highest success rate.

Model	Accuracy	MRR	H@1	H@3	H@5	H@10
CNN	0.5598	0.7173	0.56	0.85	0.93	0.98
Transformer	0.5213	0.6836	0.52	0.82	0.91	0.97
Bag-of-Emb	0.5580	0.7124	0.56	0.85	0.93	0.98
LSTM	0.5594	0.7137	0.56	0.85	0.93	0.98
GRU	0.5594	0.7137	0.56	0.85	0.93	0.98

Table 1

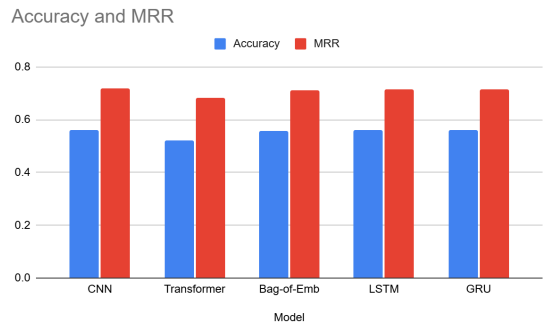

Table 2

Table 1 reporting a similar hit rate at K across the various models is specifically due to the specific runtime the models were being run on, and some models found roughly the same result due to the optimal parameters causing homogenization of results. Further research on these models is currently focused on the CNN model specifically, as qualitative results currently show that it is giving the highest success rate across a variety of parameters, the highest currently being an accuracy rate of 0.5598. Experimentation with optimization pushed accuracy rate inconsistently towards accuracy rates of 0.6, with the Adams optimizer with 32 hidden layers finding the most general success rate.

5. Discussion

Our early experiments show that simple sequence models such as 1D-CNN, LSTM, and GRU outperform Transformers on our GUI transition prediction task. This suggests that GUI interaction paths primarily rely on local, short-range dependencies (e.g., "element → action → element"), making complex self-attention architectures less advantageous. We also found that our token-based preprocessing pipeline (element/action abstraction + removal of unresolved transitions) noticeably stabilized training and improved label consistency across models. Overall, these findings indicate that architectural simplicity with strong local inductive bias is more suitable for GUI next-element prediction than large Transformers, guiding our choice to adopt CNN as the primary model for further experiments.

6. Conclusion

Our current solution for a machine learning model that is able to predict a user's path through a GUI is a CNN model that is tuned through a variety of parameters and hyperparameters to optimize our models predictive power for the next possible actions a user could take through a GUI. Experimentation on this model is proving increasingly successful, and we believe that further tuning will allow us to achieve a model that is able to accurately predict user pathing through a GUI at a highly successful rate. The success of this model has potential to lead to user-dependent GUIs, which would increase user throughput on a majority of applications. This model has the potential to optimize and vastly improve GUI at a large scale if predictive accuracy is optimized.

ACKNOWLEDGMENTS

This work is supported by Arizona State University, and done with the guidance of Professor Hua Wei in the Computer Science Department. This project was done for a final project for the course titled CSE 486, Foundations of Machine Learning. The dataset we created this model on was collected from the AutoDroid-V2 project, which was created by Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, and Yuanchun Li.

REFERENCES

[1] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhui Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, and Yuanchun Li. 2025. AutoDroid-V2: Boosting SLM-based GUI Agents via Code Generation. Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services. Association for Computing Machinery, New York, NY, USA, 223–235. https://doi.org/10.1145/3711875.3729134

[2] Seong-Guk Nam and Yeong-Seok Seo. 2023. GUI Component Detection-Based Automated Software Crash Diagnosis. Electronics, 12, 2382. https://doi.org/10.3390/electronics12112382

[3] Dang Nguyen, Jian Chen, Yu Wang, Gang Wu, Namyong Park, Zhengmian Hu, Hanjia Lyu, Junda Wu, Ryan Aponte, Yu Xia, Xintong Li, Jing Shi, Hongjie Chen, Viet Dac Lai, Zhouhang Xie, Sungchul Kim, Ruiyi Zhang, Tong Yu, Mehrab Tanjim, Nesreen K. Ahmed, Puneet Mathur, Seunghyun Yoon, Lina Yao, Branislav Kveton, Jihyung Kil, Thien Huu Nguyen, Trung Bui, Tianyi Zhou, Ryan A. Rossi, and Franck Dernoncourt. 2024. Gui Agents: A survey. arXiv preprint arXiv:2412, 13501 (Dec 2024). https://arxiv.org/abs/2412.13501

[4] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. 2022. A survey of transformers. AI open, 3, https://arxiv.org/abs/2106.04554

[5] Rie Johnson and Tong Zhang. 2015. Effective Use of Word Order for Text Categorization with Convolutional Neural Networks. In Proceedings of the 2015 conference of the North American chapter of the association for computational linguistics: human language technologies. https://arxiv.org/abs/1412.1058

[6] Duyu Tang, Bing Qin and Ting Liu. 2015. Document modeling with gated recurrent neural network for sentiment classification. In Proceedings of the 2015 conference on empirical methods in natural language processing. https://10.18653/v1/D15-1167

[7] Sebastian Ruder. An overview of gradient descent optimization algorithms. 2016. arXiv preprint arXiv:1609.04747

[8] Yoon Kim. 2014. Convolutional neural networks for sentence classification. arXiv preprint arXiv:1408.5882

[9] Ziwei Wang, Weizhi Chen, Leyang Yang, Sheng Zhou, Shengchu Zhao, Hanbei Zhan, Jiongchao Jin, Liangcheng Li, Zirui Shao, and Jiajun Bu. 2025. Mp-gui: Modality perception with mllms for gui understanding. In Proceedings of the Computer Vision and Pattern Recognition Conference. https://arxiv.org/abs/2503.14021

[10]Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. arXiv preprint arXiv:1711.0510

[11]Ilya Loshchilov and Frank Hutter. 2016. Sgdr: Stochastic gradient descent with warm restarts. arXiv preprint arXiv:1608.03983

[12] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. 2016. Rethinking the inception architecture for computer vision. In Proceedings of the IEEE conference on computer vision and pattern recognition.

[13]Alex Graves. 2012. Long short-term memory. Supervised sequence labelling with recurrent neural networks. pp.37-45

[14]Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. Advances in neural information processing systems.

[16] Ye Zhang and Byron Walalce. 2017. A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification. In Proceedings of the Eighth International Joint Conference on Natural Language Processing, 1. https://aclanthology.org/I17-1026/

[17] Ellen M. Voorhees and Dawn M.Tice. 1999. The TREC-8 Question Answering Track Evaluation. In TREC

[18] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. Advances in neural information processing systems.

[19] Bao N. Nguyen, Bryan Robbins, Ishan Banerjee, and Atif Memon. 2014. GUITAR: an innovative tool for automated testing of GUI-driven software. Automated Software Engg. 21, 1 (March 2014), 65–105. https://doi.org/10.1007/s10515-013-0128-9

[20] Jieshan Chen, Mulong Xie, Zhenchang Xing, Chunyang Chen, Xiwei Xu, Liming Zhu, and Guoqiang Li. 2020. Object detection for graphical user interface: old fashioned or deep learning or a combination? In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020). Association for Computing Machinery, New York, NY, USA, 1202–1214. https://doi.org/10.1145/3368089.3409691

[21] Xin Zhou and Yang Li. 2021. Large-Scale Modeling of Mobile User Click Behaviors Using Deep Learning. In Fifteenth ACM Conference on Recommender Systems (RecSys '21), September 27-October 1, 2021, Amsterdam, Netherlands. ACM, New York, NY, USA 11 Pages. https://doi.org/10.1145/3460231.3474264

[22] Martin Popel, Ondřej Bojar. 2018. Training Tips for the Transformer Model. arXiv:1804.00247. Retrieved from https://arxiv.org/abs/1804.00247

[23] Xavier Amatriain, Ananth Sankar, Jie Bing, Praveen Kumar Bodigutla, Timothy J. Hazen, Michaeel Kazi. 2023. Transformer models: an introduction and catalog. arXiv:2302.07730. Retrieved from https://arxiv.org/abs/2302.07730

[24] Divyani Gupta, Namrata Dhanda, and Kapil Kumar Gupta. 2025. A comparative study of various LSTM models for stock market time series classification. 2025 8th International Conference on Computing Methodologies and Communication (ICCMC) (July 2025), 1–6. DOI:http://dx.doi.org/10.1109/iccmc65190.2025.11140645

[25] Cem Tutum, Suhaib AbdulQuddos, and Risto Miikkulainen. 2021. Generalization of Agent Behavior through Explicit Representation of Context. In 2021 IEEE Conference on Games (CoG), 1-7. DOI: 10.1109/COG52621.2021.9619141 Retrieved from https://ieeexplore.ieee.org/abstract/document/9619141

[26] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2024. OS-Atlas: A Foundation Action Model for Generalist GUI Agents. arXiv preprint arXiv:2410.23218. Retrieved from https://arxiv.org/abs/2410.23218